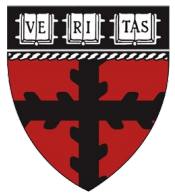
Lecture 2: Containers I

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Logistics / Reminders

1. DCE waitlisted students were **admitted**. Please look at the post on Ed.

2. DCE students online: we know the chat on Zoom is not working, so please go to Ed and find chat on the top right. See **lecture2** and use that for interacting with the teaching team.
   https://edstem.org/us/courses/97373/discussion/chat/channels/30245

3. Please fill this survey.
   https://bit.ly/ac215_2026_survey



4. When you form a team, please fill this form too.
   https://bit.ly/ac215_2026_groups



Outline

1. Recap & Motivation
2. What is a Container
3. Why use Containers
4. How to use Containers

Outline

1. **Recap & Motivation**
2. What is a Container
3. Why use Containers
4. How to use Containers



Recap Virtual Machines: Pros and Cons

Pros

- **Full Autonomy:**
  Complete control over the operating system and applications, similar to a physical server.
- **Very Secure:**
  Isolated environment helps in minimizing the risk of system intrusion.
- **Lower Cost:**
  Can be more cost-effective for applications that need full OS functionality.
- **Cloud Adoption:**
  Offered by all major cloud providers for on-demand server instances.

Cons

- **Resource Intensive:**
  Consumes hardware resources from the host machine.
- **Portability Issues:**
  VMs are large in size, making them harder to move between systems.
- **Overhead:**
  Requires additional resources to run the hypervisor and manage multiple operating systems.

Recap: Virtual Environments: Pros and Cons

Pros

- Reproducible Research:
  - Easy to replicate experiments and share research outcomes due to consistent environments.
- Explicit Dependencies:
  - Clear listing of all required packages and versions, reducing ambiguity.
- Improved Engineering Collaboration:
  - Team members can quickly set up the same environment, streamlining development.

Cons

- Difficulty in Setup:
  - Initial setup can be complex, especially for those new to the concept.
- No Isolation from Host:
  - Virtual environments share the host's operating system, leading to potential conflicts.
- OS Limitations:
  - May not be compatible across different operating systems, requiring additional configuration.

Wish List

Automated Setup:

Automatically set up (installs) OS and extra libraries and set up the python environment.

Isolation:

Complete separation from the host machine, ensuring a consistent run-time environment.

Resource Efficiency:

Minimal use of CPU, Memory, and Disk resources, optimized for performance.

Quick Startups:

Near-instantaneous initialization, reducing time to deployment.

Containers

Outline

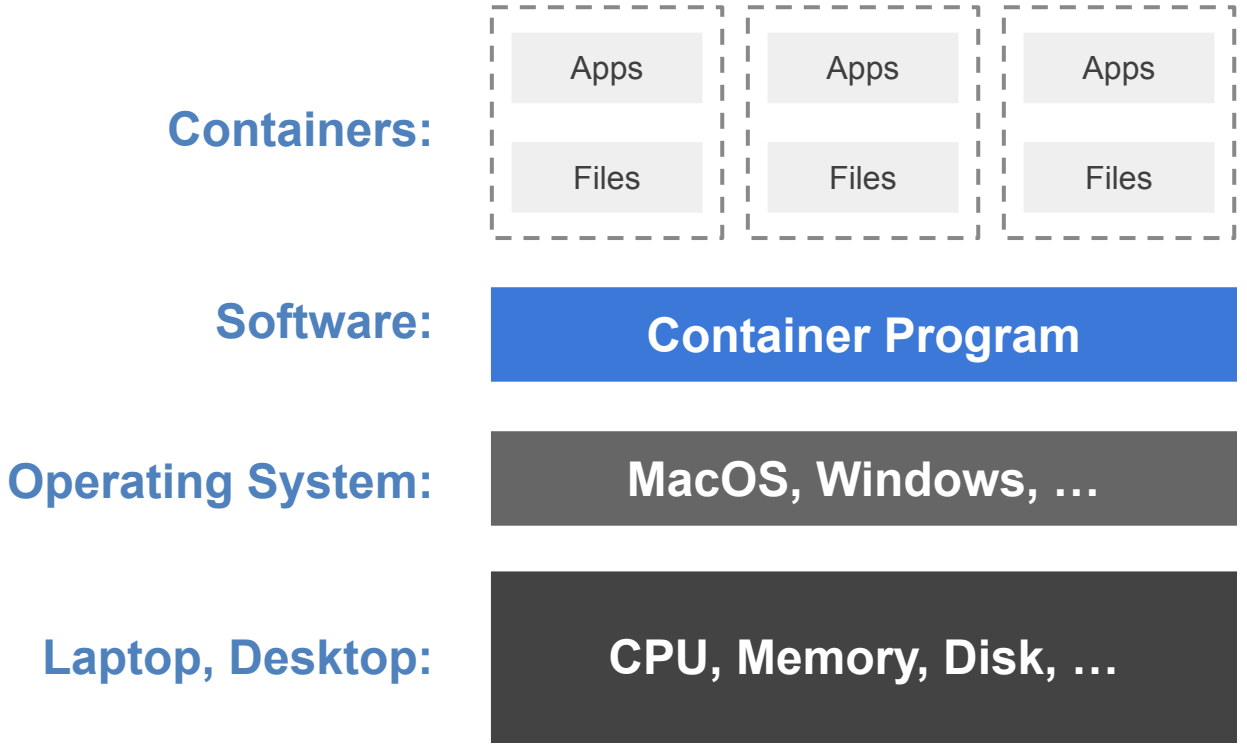
1. Recap & Motivation
2. **What is a Container**
3. Why use Containers
4. How to use Containers



What is a CONTAINER

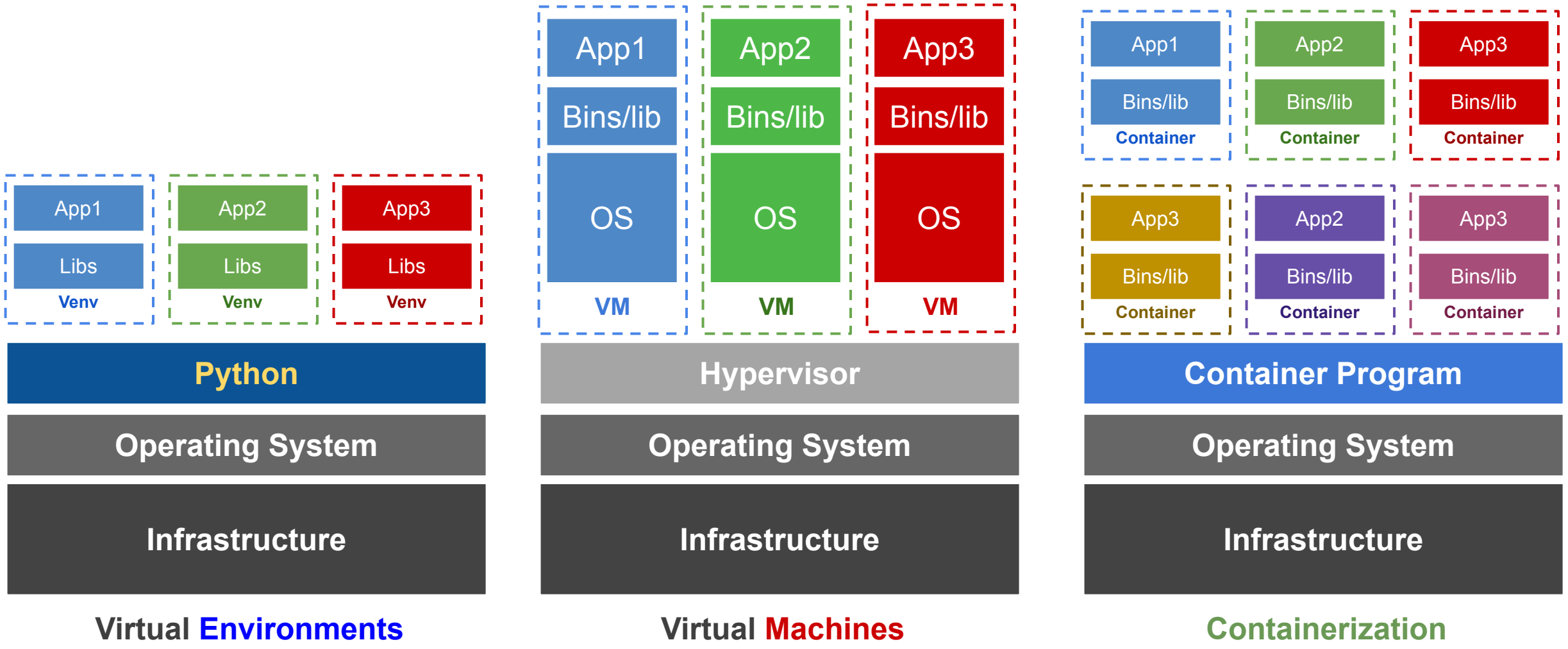
A container is a program that runs on your host machine — like a mini computer inside it.

It shares the host's CPU, memory, and disk, but keeps its own OS, file system, and network.

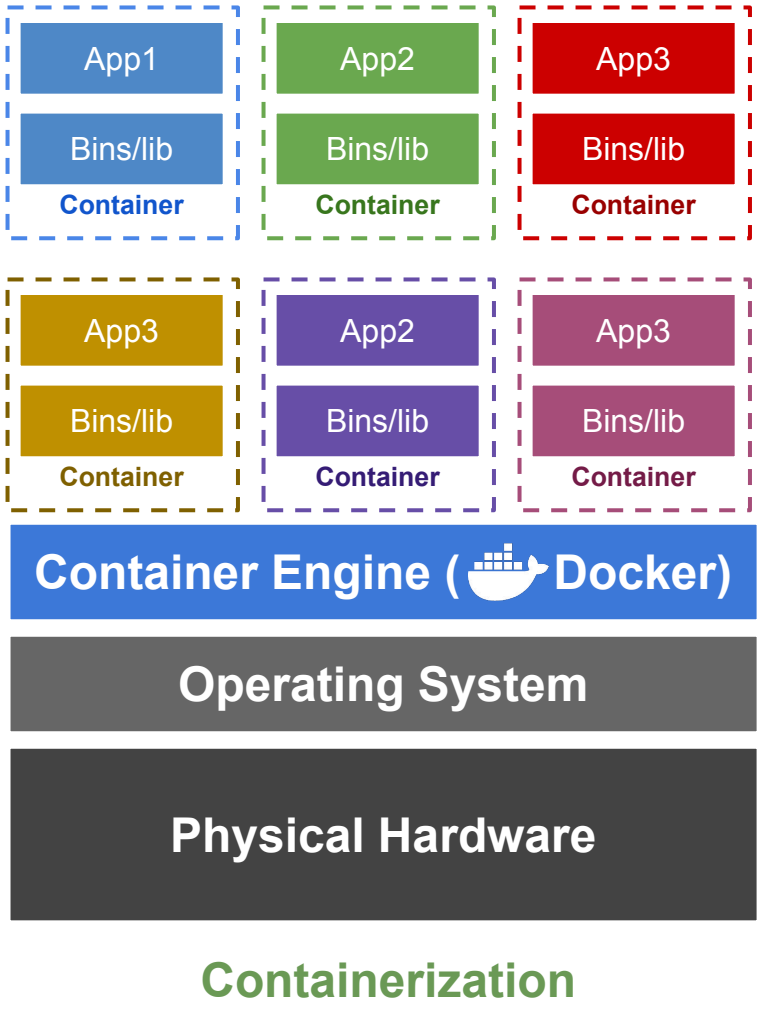


It packages code with its dependencies, so it runs the same no matter where it's deployed.

Environments vs Virtualization vs Containerization



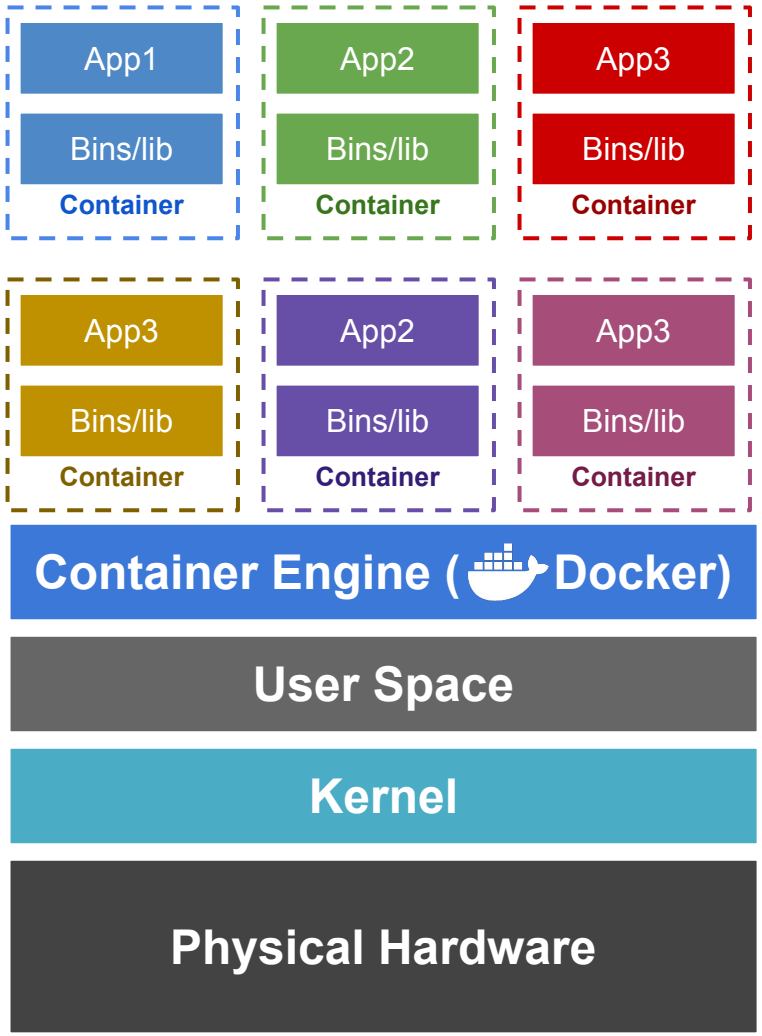
Containerization



Containerization

To understand how containers work, we need to first introduce two key Linux kernel features: namespaces and cgroups.

Containerization: Namespaces

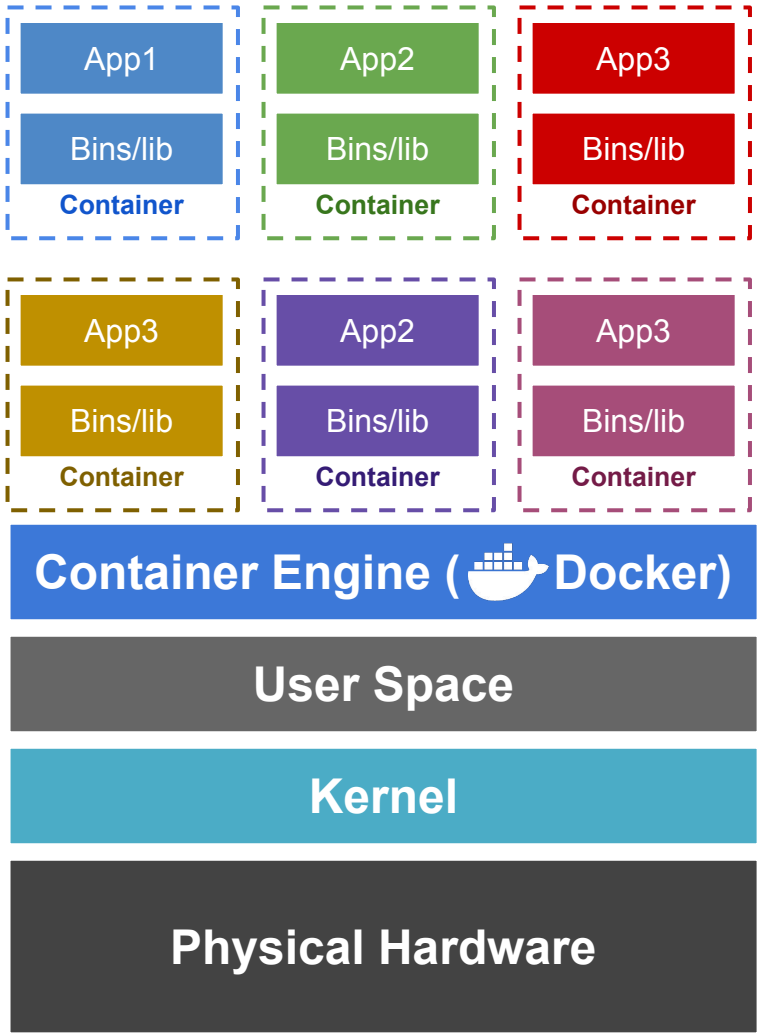


Containerization

To understand how containers work, we need to first introduce two key Linux kernel features: namespaces and cgroups.

The Operating System contains the **Kernel**, which has low level access to the hardware and the **User Space** which contains programs outside the Kernel.

Containerization: Namespaces



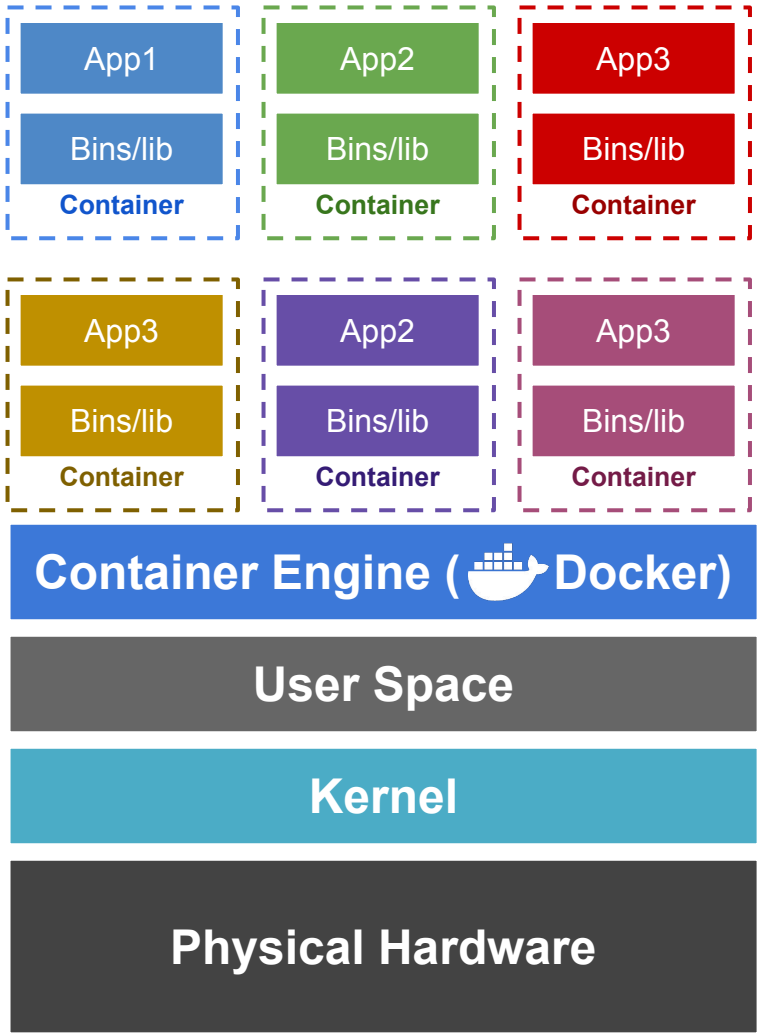
Containerization

Namespace is a feature provided by the Linux Kernel that creates an isolation between system components.

Namespaces allow different processes (or groups of processes) to have their own separate view of system resources, such as process IDs, file systems, network interfaces, and more.

When a process is placed into a namespace, it can only see and interact with the resources within that namespace.

Containerization: PID Namespaces

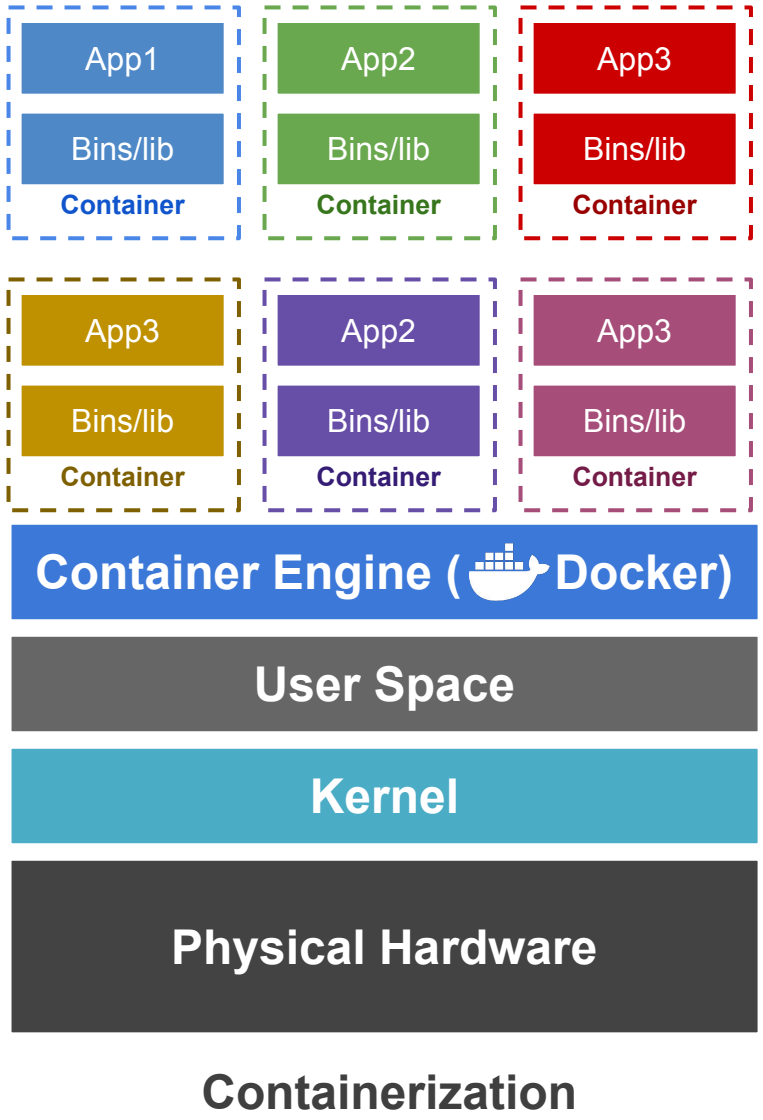


Containerization

- **PID Namespace:** processes inside different PID namespaces can have the same process ID (PID) Each namespace starts its own PID numbering from 1. The host system sees all processes with unique global PIDs in its own namespace.

Example: two containers running on the same host. In one container, a web server process (e.g., Nginx) might have a PID of 1. In another container, a database process (e.g., MySQL) could also have a PID of 1. On the host, these processes could be assigned global PIDs 345 and 678, respectively.

Containerization: Mount Namespaces

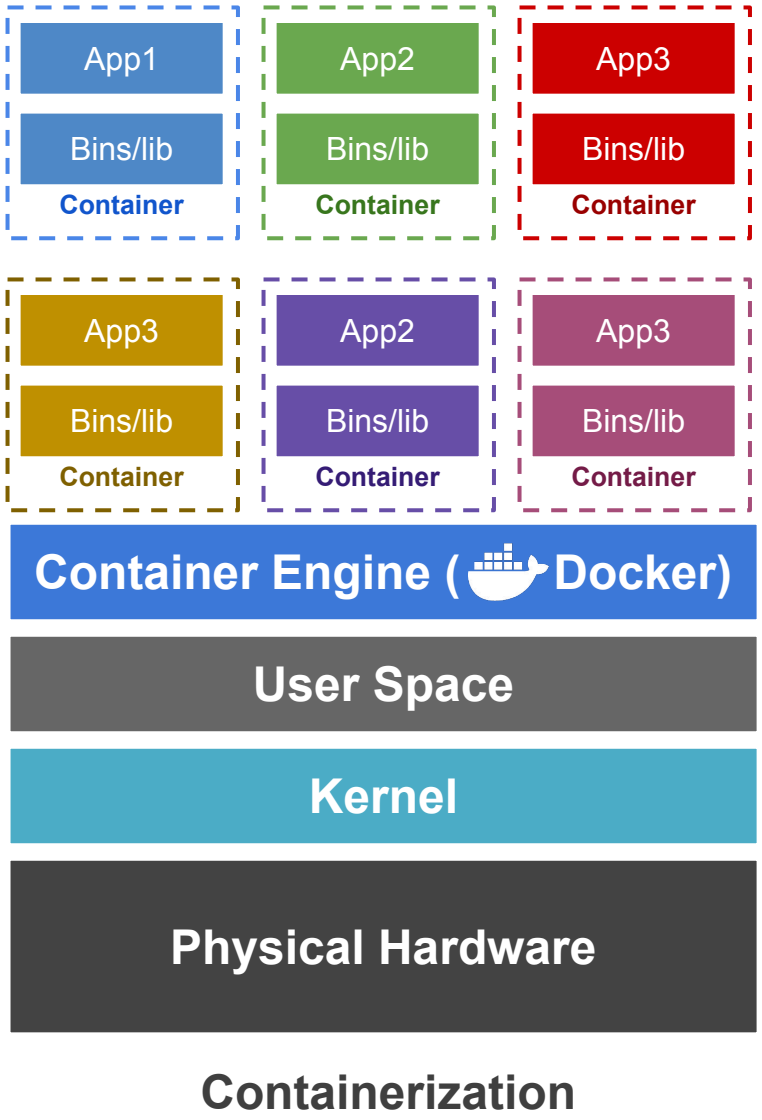


Containerization

- **Mount Namespace:** different containers will have their own view of the filesystem. This includes mounted disks, mount points or directories.

  Example: Suppose you have two containers that mount to the directory /data, where one container mounts *Drive_A* and the other *Drive_B*. Both containers will be unaware of other mounted drives.

Containerization: Network Namespaces

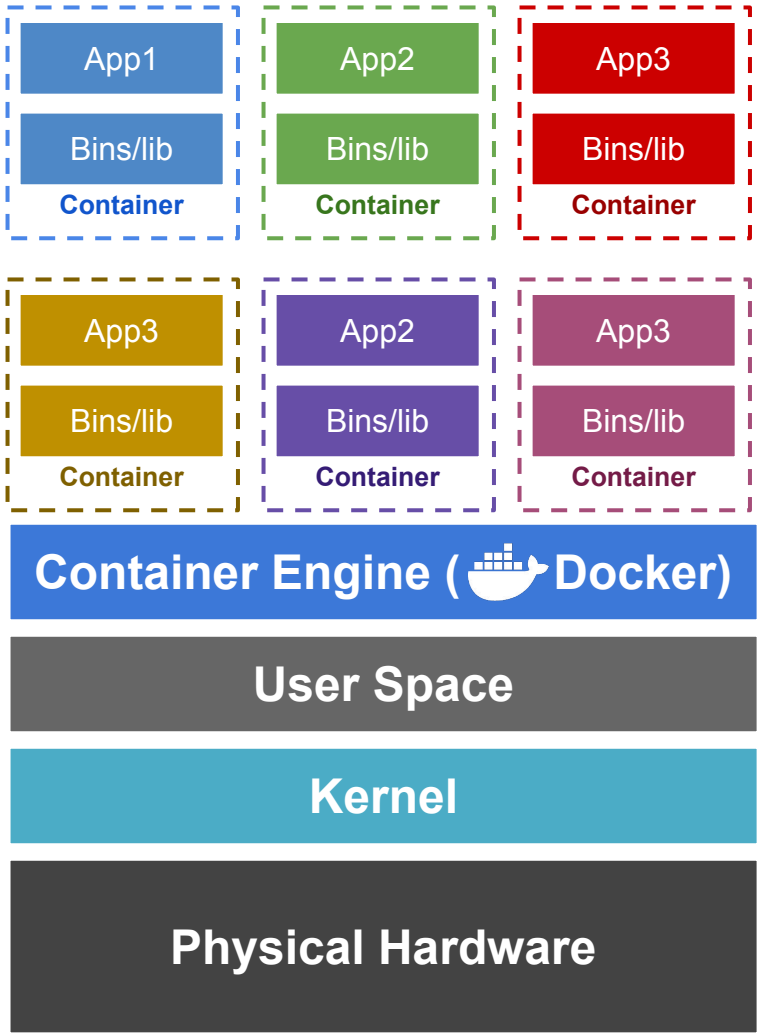


Containerization

- **Network Namespace:** different containers will have their own isolated sub-network to interact with. This include IP addresses, routing tables, firewall rules, etc.

  Example: multiple containers can use the same IP address to perform tasks, without worrying about interfering with each other or security concerns.

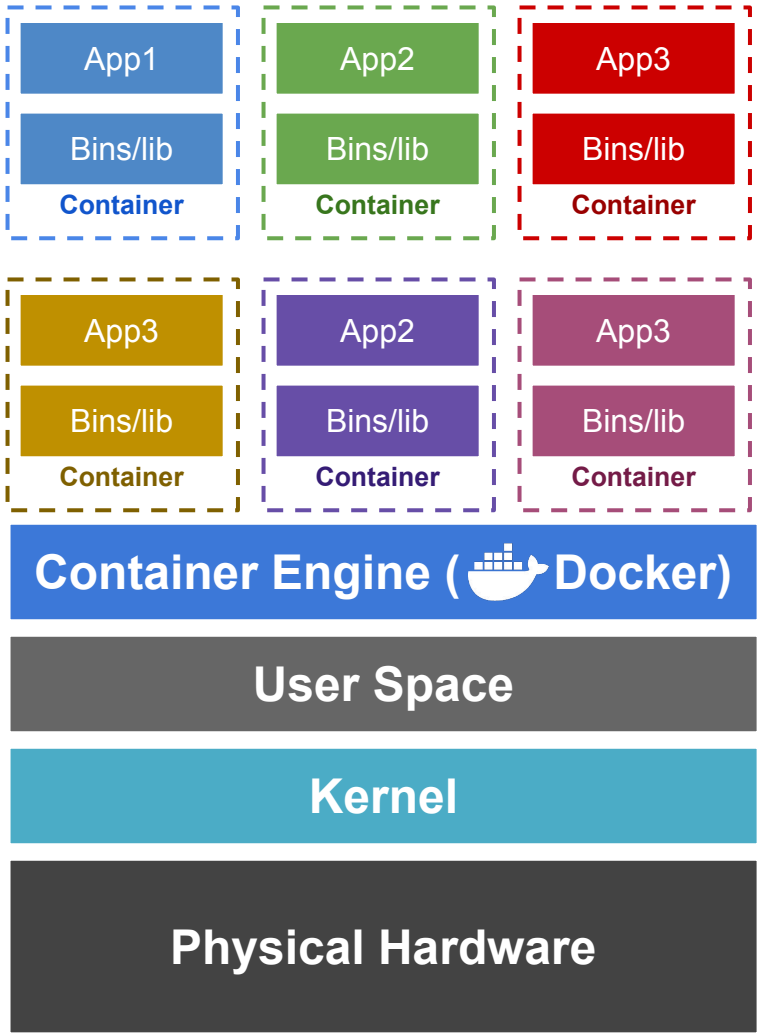
Containerization: Namespaces



Containerization

- **IPC Namespace:** Inter-Process Communication. It isolates the processes communication and shared memory within each namespace.
- **UTS Namespace:** UNIX Timesharing System. Allows each namespace to define its own localhost.
- **User Namespace:** Isolates user groups within each container. Even if a process is running as *root* inside the container, it will have *non-root* privileges in the host.

Containerization: Control Groups



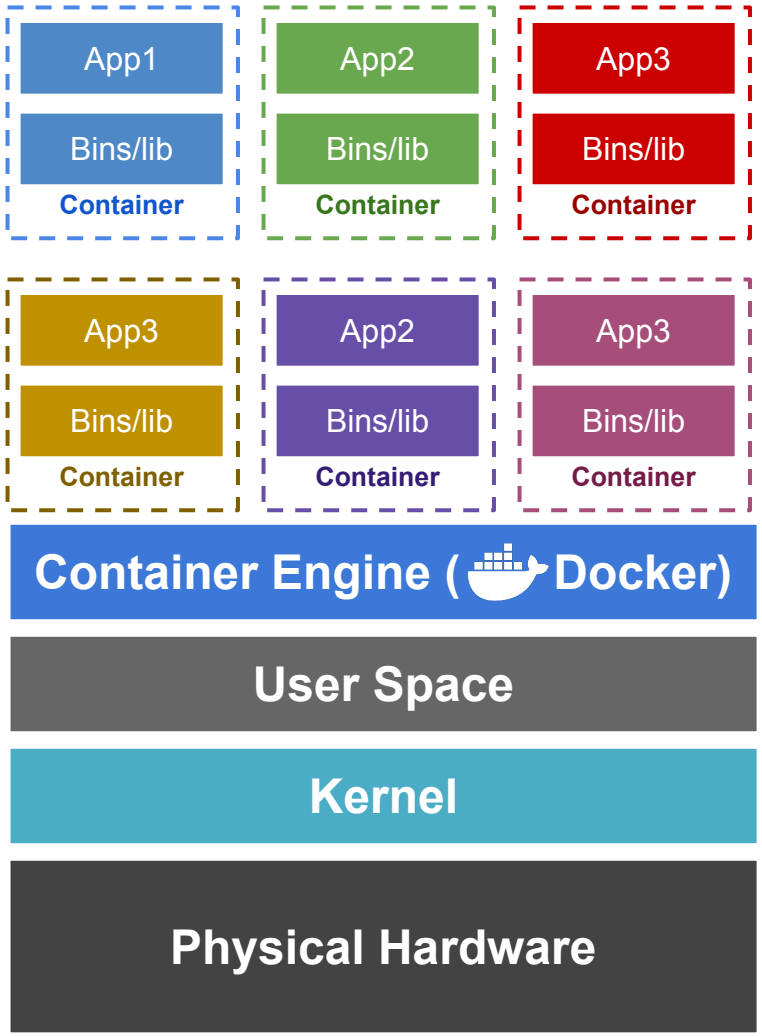
Containerization

Complementary to namespaces, cgroups (Control Groups) allow the limitation and management of system resources such as CPU, memory, disk I/O, and network bandwidth.

By controlling resource allocation, cgroups enable more efficient resource utilization and isolation within containers, making them more lightweight and flexible compared to virtual machines (VMs).

Also, they provide an additional layer of security ensuring that one container cannot bring the system down by exhausting one of those resources.

Containerization: Security Features



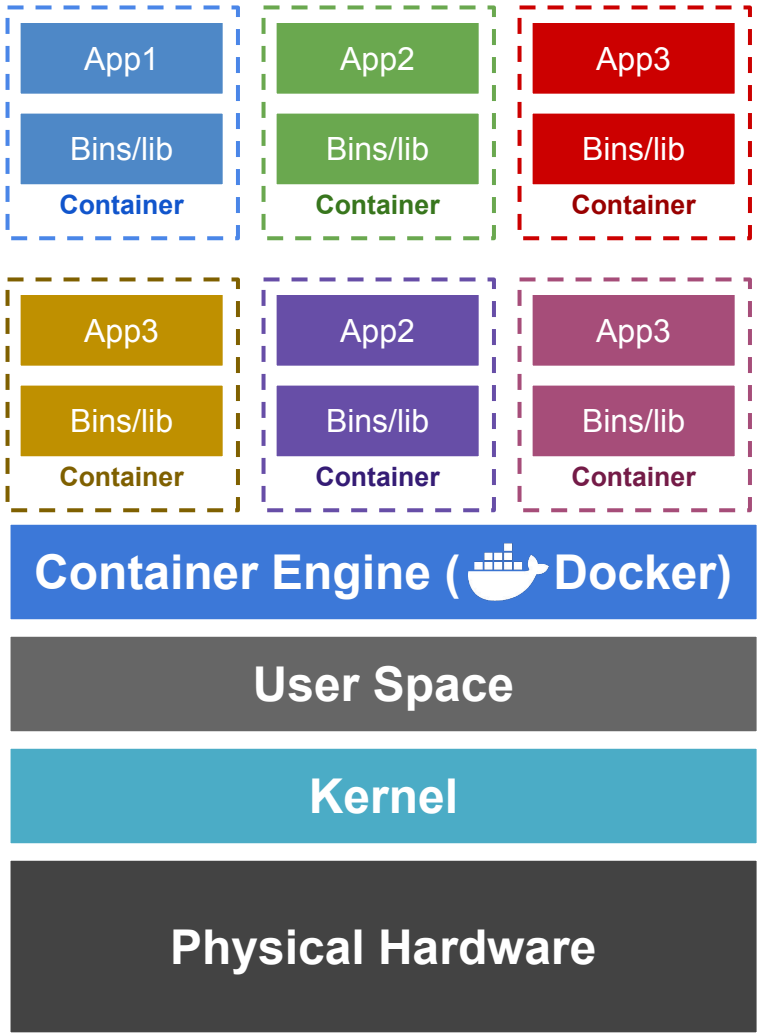
Containerization

Apart from namespaces and cgroups, the Docker engine utilizes additional kernel features to increase security.

By default, containers are given a reduced set of privileges (Secure computing mode, seccomp) reducing by 44 the available system calls (300+). This ensures that containers remain isolated and cannot control the host.

A container is unlikely to require root privileges, since those tasks can be executed by the host. Only the absolutely necessary information is passed into the container.

Containerization: Security Features



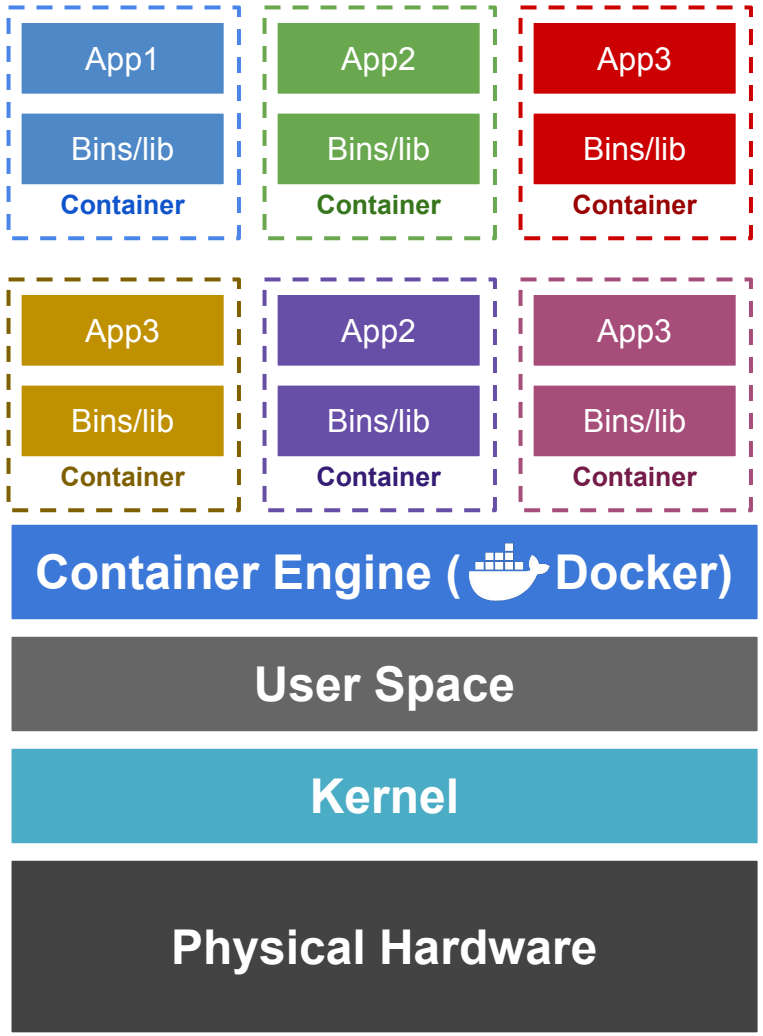
Containerization

Example:

Unless configured otherwise, the containers don't have access to the *syscall reboot*. Which would allow a container to reboot the system.

If a container has access to the syscall *quotactl*, it would have the ability to change the disk quotas, affecting the rest of the host and other containers or VMs.

Containerization: Security features



Containerization

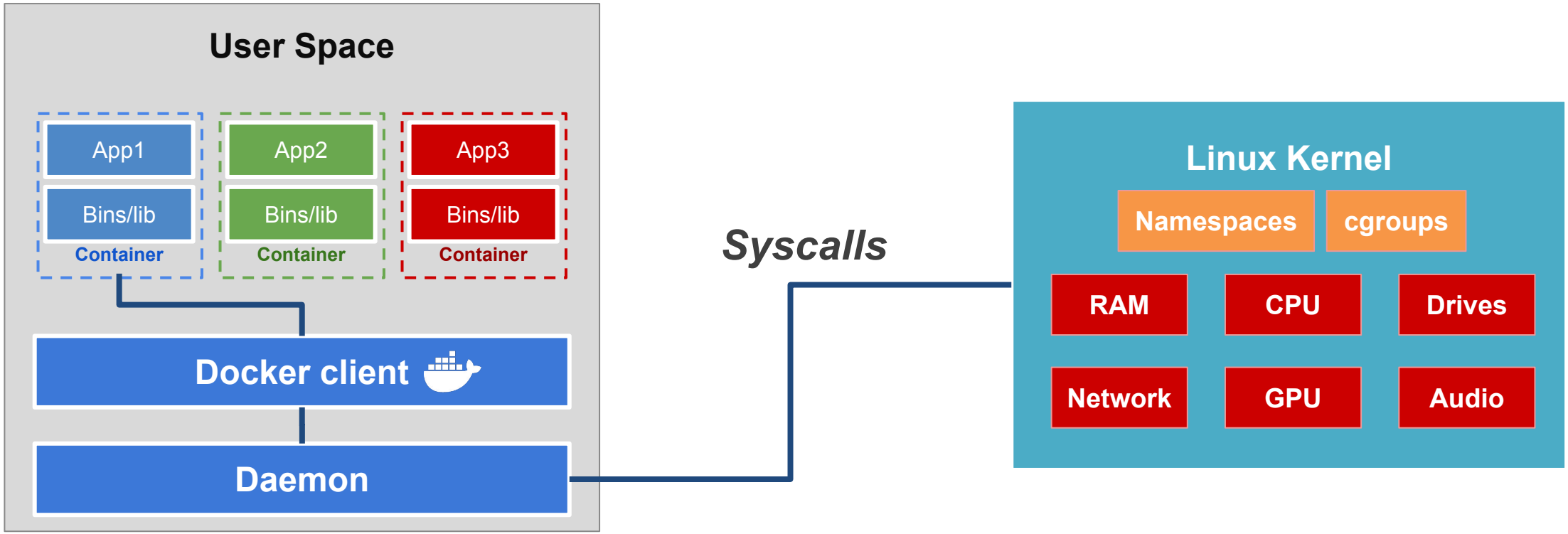
Additional security layer are applied, such as AppArmor (Application Armor) and SELinux (Security-Enhanced Linux).
These modules restrict the the usage of files, directories, sockets, and other processes. Providing an additional layer of security, preventing any container of accessing core system components.

Do not confuse with cgroups, which control resource management!

What Makes Containers so Small?

Container = User Space of OS

Each container has the minimum code required to run its program. It leverages the host Os (User Space and Kernel) to perform its task.



Containers compatibility

If Docker containers rely on Linux kernel features, how can we use them on MacOS and Windows?

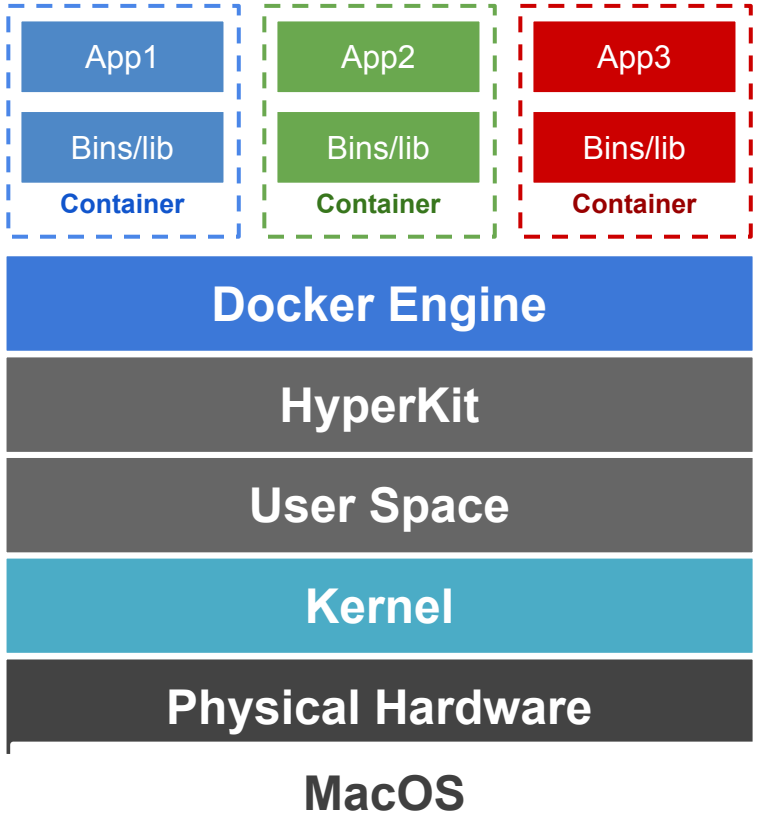


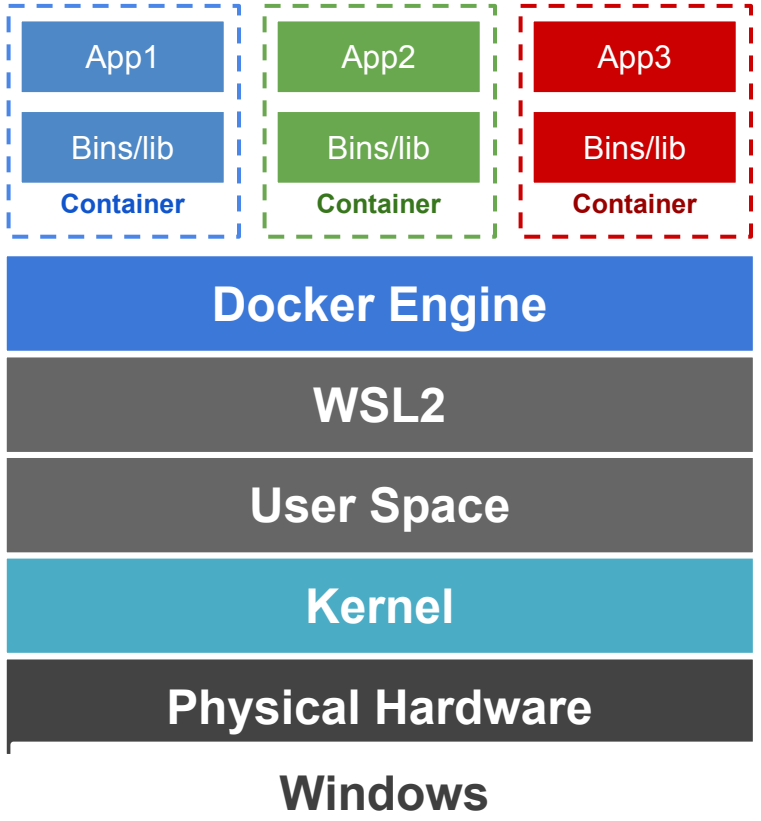
Both OS's spin stripped down VMs that translate *syscalls* from the Daemon to the native Kernel.

Containers compatibility

If Docker containers rely on Linux kernel features, how can we use them on MacOS and Windows?

Both OS's spin stripped down VMs that translate *syscalls* from the Daemon to the native Kernel.





HyperKit is a virtualization technology.

WSL2 is a custom built Linux kernel, integrated with Windows.
It spins an even thinner Linux VM, which allows it to run native linux programs.

Outline

1. Recap & Motivation
2. What is a Container
3. **Why use Containers**
4. How to use Containers

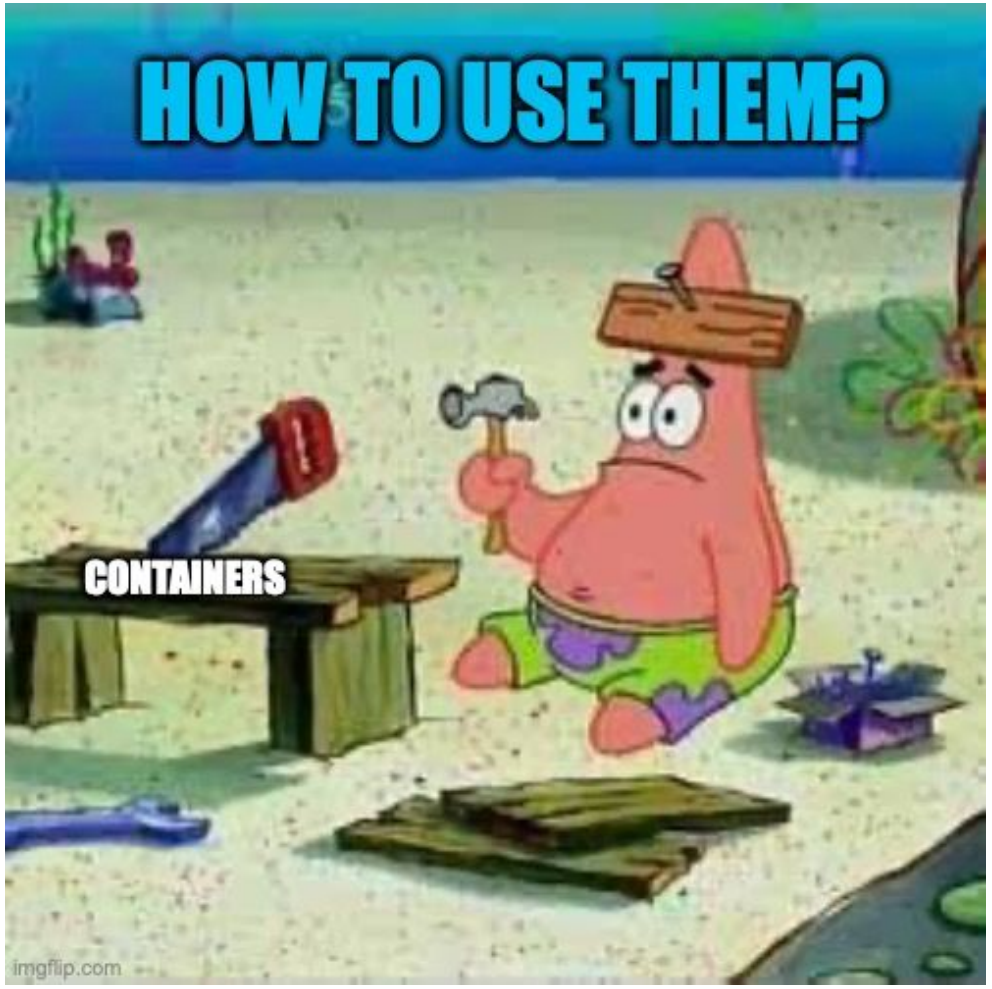


Advantages of a CONTAINER

- **Portability & Lightweight:** Containers encapsulate everything needed to run an application, making them easy to move across different environments.
- **Fully Packaged**: Containers include the software and all its dependencies, ensuring a consistent environment throughout the development lifecycle.
- **Versatile Usage:** Containers can be used across various stages, from development and testing to training and production deployment

Outline

1. Recap & Motivation
2. What is a Container
3. Why use Containers
4. **How to use Containers**



Examples of Containerization Technologies:

- **LXC (Linux Containers):** The original, lightweight Linux container technology; offers less isolation than Docker.
- **Podman:** A daemonless, Docker-compatible container engine prioritizing security, including rootless execution.
- **rkt (Rocket):** A security-focused, simple, and composable alternative to Docker.
- **Orbstack:** A fast, lightweight, macOS-exclusive platform for container and VM development.
- **Docker:** The industry-standard container platform, noted for its ease of use, vast ecosystem, and robust support.



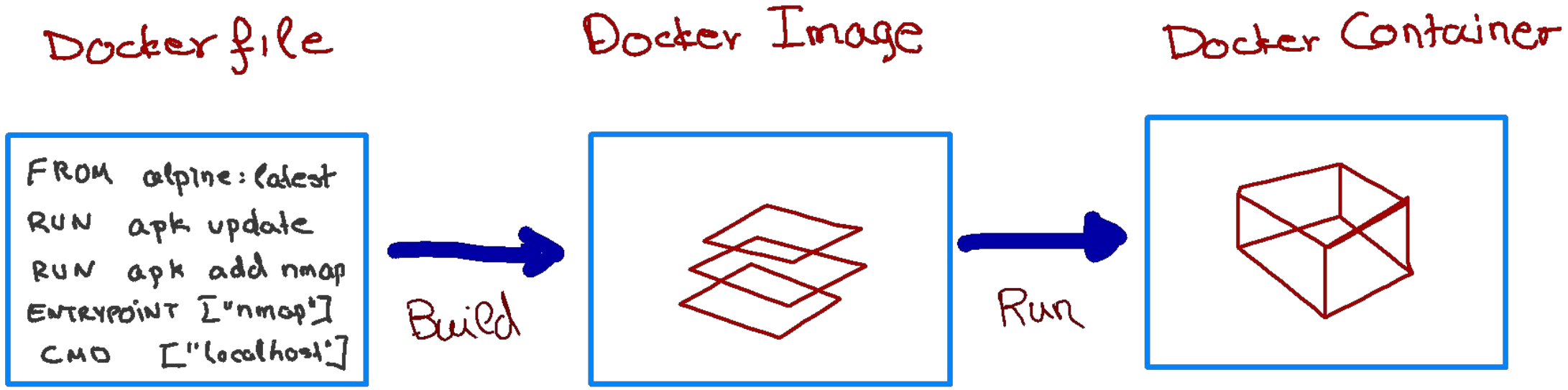
What is docker?

- **Open Source:** A community-driven, highly compatible approach to software development.
- **Platform Functionality:** Simplifies the process of developing, shipping, and running applications through containerization.
- **Portability:** Ensures applications perform consistently across diverse computing environme
- **Robust Ecosystem:** Supported by widely used tools and resources, including Docker Hub and Kubernetes.



From Dockerfile to Container

- We use a simple text file, the Dockerfile, to build the Docker Image, which consists of an iso file and other files.
- We run the Docker Image to get Docker Container.



What is the difference between an image and container

A Docker image functions as a template or blueprint used to launch a running Docker container, providing the necessary configuration to instantiate it.

To visualize the relationship between these concepts, consider this analogy:

- **Docker file:** The handwritten recipe description.
- **Docker image:** A pre-assembled ingredient kit (like a meal delivery service).
- **Docker container:** The final, cooked dish.

Alternatively, you can view the relationship through an object-oriented programming lens: the image acts as the class, while the container functions as an instance of that class.

Anatomy of a Dockerfile



FROM: Specifies the base OS image (e.g., alpine, Ubuntu) for building the Docker image.

RUN: Executes commands to build the image. Each **RUN** creates a new layer.

ENTRYPOINT: Sets the default executable for the container, making it behave like a standalone application.

CMD: Sets default commands or parameters for container startup, but can be overridden by the `docker run` command.

https://docs.docker.com/engine/reference/builder/

Anatomy of a Dockerfile

FROM: Specifies the base OS image (e.g., alpine, Ubuntu) for building the Docker image.

RUN: Executes commands to build the image. Each RUN creates a new layer.

ENTRYPOINT: Sets the default executable for the container, making it behave like a standalone application.

CMD: Sets default commands or parameters for container startup, but can be overridden by the `docker run` command.

ADD: Similar to **COPY**, but can also handle URLs and auto-extract compressed files.

ENV: Sets environment variables within the Docker image. These variables can be used in subsequent commands or by applications within the container.

WORKDIR: Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD instructions that follow in the Dockerfile.

https://docs.docker.com/engine/reference/builder/

Docker Image as Layers

When we execute the `build` command, the daemon reads the `Dockerfile` and creates a layer for every command.

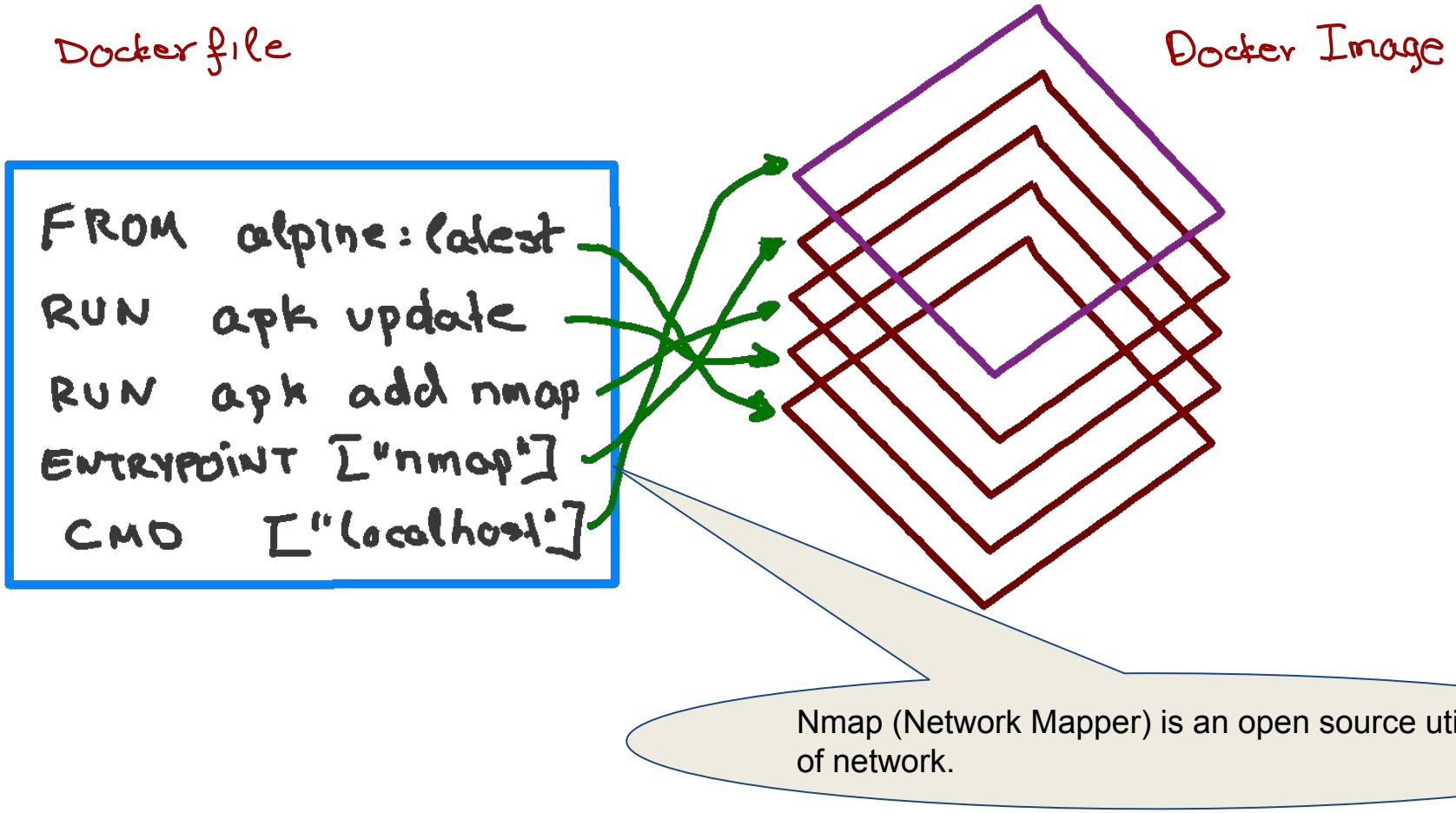
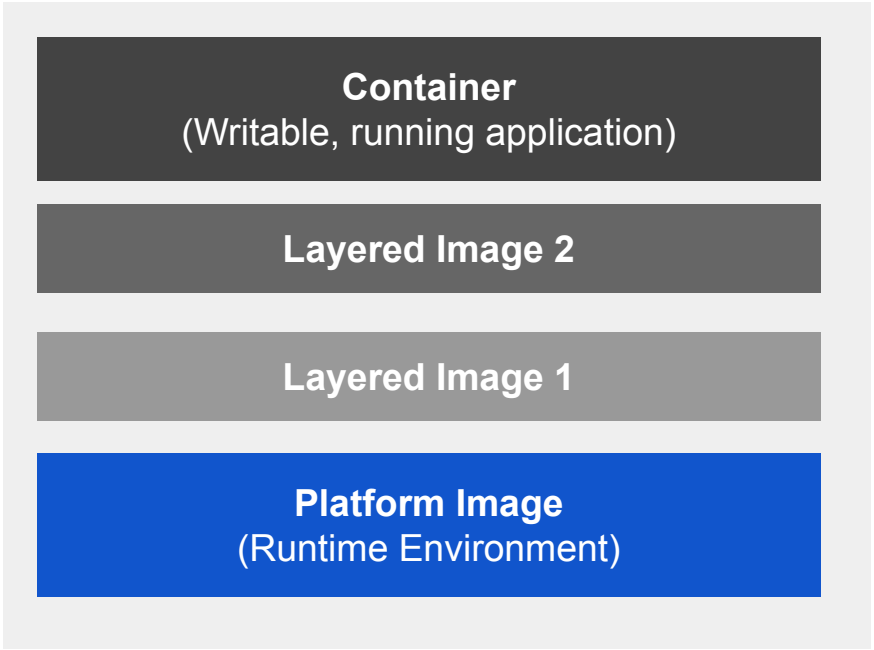


Image Layering



A application sandbox
- Each container is based on an image that holds necessary config data
- When you launch a container, a writable layer is added on top of the image

A static snapshot Images are read-only and capture the container's settings.
- Layer images are read-only
- Each image depends on one or more parent images

Platform images define the runtime environment, packages and utilities necessary for containerized application to run. It is an Image that has no parent

Why Layers

Why build an image with multiple layers when we can just build it in a single layer?

Efficiency

Reuse common layers across different images, saving storage and speeding up image creation.

Incremental Updates

Update only the changed layer, reducing the time and bandwidth needed for deployment.

Cache Utilization

Docker caches layers. If no changes are detected, subsequent builds are faster.

Modularity

Break down complex setup into manageable pieces, making debugging easier.

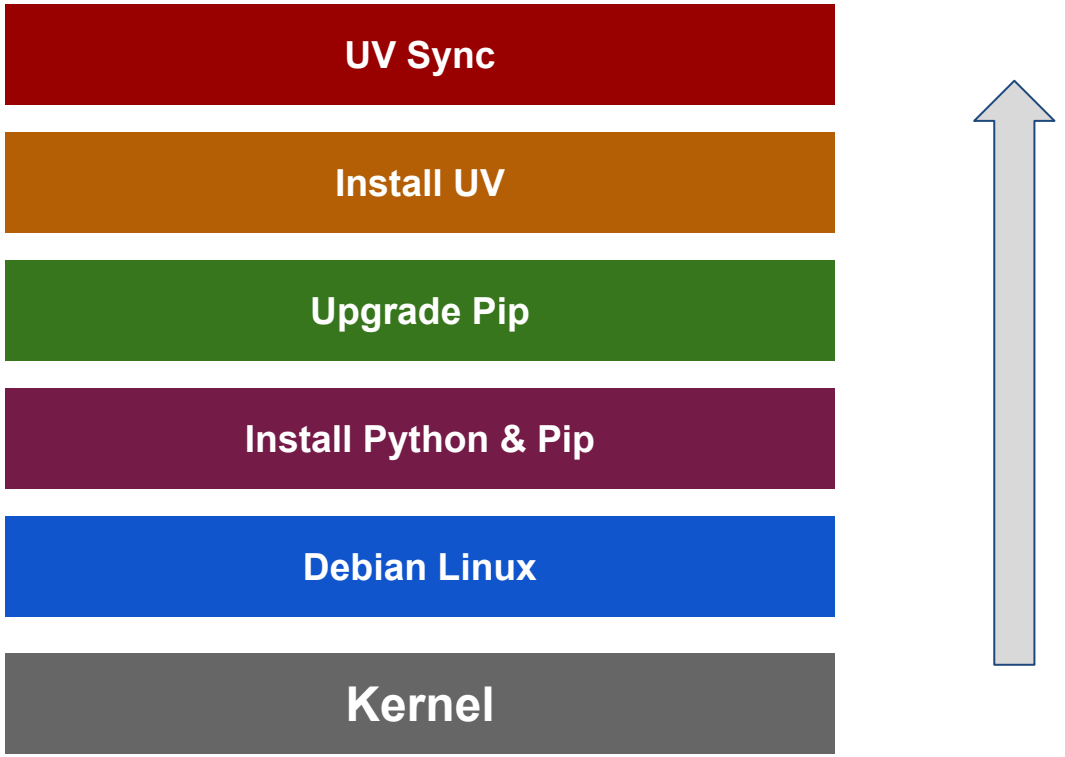
Security

Smaller attack surface per layer and easier to scan for vulnerabilities.

WE WILL SEE AN EXAMPLE LATER

Image Layering - Example

Docker layers for a container running debian and a python environment using UV



Docker Vocabulary



Docker File
A text document with commands on how to create an Image



Docker Image
The basis of a Docker container. Represent a full application

Images
How you **store** your application



Docker Container
The standard unit in which the application service resides and executes

Containers
How you **run** your application



Docker Engine
Creates, ships and runs Docker containers deployable on a physical or virtual, host locally, in a datacenter or cloud service provider

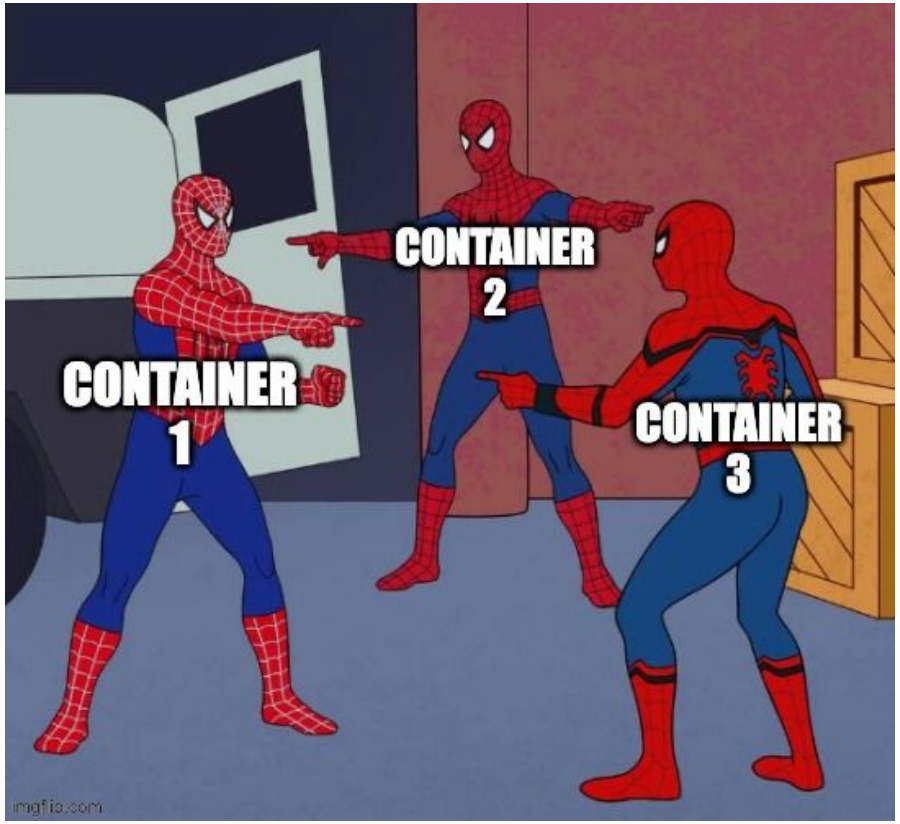


Registry Service (Docker Hub or Docker Trusted Registry)
Cloud or server-based storage and distribution service for your images

FAQ: Running Multiple Containers from a Single Image

How can you run multiple containers from the same image?

Yes, you could think of an image as instating a class. You can create multiple instances (containers) from a single image.



FAQ: Running Multiple Containers from a Single Image

How can you run multiple containers from the same image?

Yes, you could think of an image as instating a class. You can create multiple instances (containers) from a single image.

Wouldn't all these containers be identical?

Not necessarily. Containers can be instantiated with different parameters using the CMD command, making them unique in behavior.



FAQ: Running Multiple Containers from a Single Image

How can you run multiple containers from the same image?

Yes, you could think of an image as instating a class. You can create multiple instances (containers) from a single image.

Wouldn't all these containers be identical?

Not necessarily. Containers can be instantiated with different parameters using the CMD command, making them unique in behavior.

Dockerfile

```
FROM ubuntu:latest
RUN apt-get update
ENTRYPOINT ["/bin/echo", "Hello"]
CMD ["world"]
```

```
> docker build -t hello_world_cmd -f Dockerfile .

> docker run -it hello_world_cmd
> Hello world
> docker run -it hello_world_cmd Pavlos
> Hello Pavlos
```

Mounting

Containers are **ephemeral** by nature. Because a container's filesystem operates as a writable layer, any changes made—especially when using the `--rm` flag—are discarded once the container exits.

This creates two critical requirements:

1. **Development:** The ability to edit code on the host machine while seeing those changes reflected live inside the container.
2. **Persistence:** The need to save data, such as models and outputs, so they remain available after the container is shut down.

The solution is to use **mounts**, which link host directories or Docker-managed storage directly into the container.

Mounting:

Three ways to mount

Type	Who picks the location	Persists?	Typical use
Bind mount	you (a host path)	yes but only on host	source code, config, secrets (dev)
Named volume	Docker	yes (Docker-managed)	db files, model artifacts
tmpfs	— (RAM)	no	ephemeral secrets / scratch

Mounting: Bind mount

How it works:

Use bind mounts to map a host directory directly into a container. This allows you to edit files on your local machine and see the changes reflected instantly inside the container, eliminating the need for constant rebuilds.

Command Syntax:

```
docker run -v "$(pwd)":/app <image_name>
```

This maps your current working directory on the host to `/app` inside the container.

Mounting: Bind mount (cont)

Use Cases:

- **The Dev Loop:** Essential for active development

- **Secret Management:** Inject sensitive configuration files securely by mapping them into the container:

```
-v "$(pwd)/secrets":/secrets
```

Note: You must always provide an absolute path for the host directory; `$(pwd)` is a reliable way to generate this.

Mounting: Named Volumes (cont)

Key Commands:

- Create a volume:

  ```
  docker volume create mydata
  ```

- Mount volume to container:

  ```
  docker run -v mydata:/data transcribe-audio
  ```

- Manage volumes:

  `docker volume ls` (list)

  `docker volume inspect` (examine),

  `docker volume rm` (delete)

Mounting: Named Volumes (cont)

Core Concepts:

- **Persistence:** Data remains intact even after the container is removed.
- **Shared Access:** Volumes can be mounted simultaneously across multiple containers.
- **Managed Storage:** Docker handles the underlying storage location on the host system.

Mounting: Named Volumes (cont)

Best Use Cases:

- Storing database files.
- Persisting machine learning model artifacts.
- Retaining any critical data that needs to outlive the container lifecycle.

Mounting: bind mount or named volume?

Docker uses the `-v` flag to distinguish between bind mounts and named volumes based on the presence of a forward slash (`/`).

- **Bind Mounts:** If the source path contains a `/`, Docker maps a specific directory from your host machine (e.g., `docker run -v "$(pwd)":/app app`). You control the location.
- **Named Volumes:** If the source path does not contain a `/`, Docker treats it as a managed volume name, creating it if necessary (e.g., `docker run -v appdata:/data app`). Docker manages the storage.

Both persist data after container exit, but bind mounts utilize a host-controlled path, while named volumes rely on Docker-managed storage.

Mounting: -v vs --mount

- `-v` is the terse legacy form; `--mount` is explicit and safer:

```
--mount type=bind,source="$(pwd)",target=/app
--mount type=volume,source=mydata,target=/data
```

- Gotcha: if the bind source is missing, --mount **errors loudly**; -v silently creates an empty dir.

Mounting: Usage

When to use which (summary)

- Bind mount → live code, config, secrets (dev). Named volume → persistent/shared data. tmpfs → ephemeral secrets.
- Read-only mounts: append :ro (e.g. secrets) — least privilege for files.

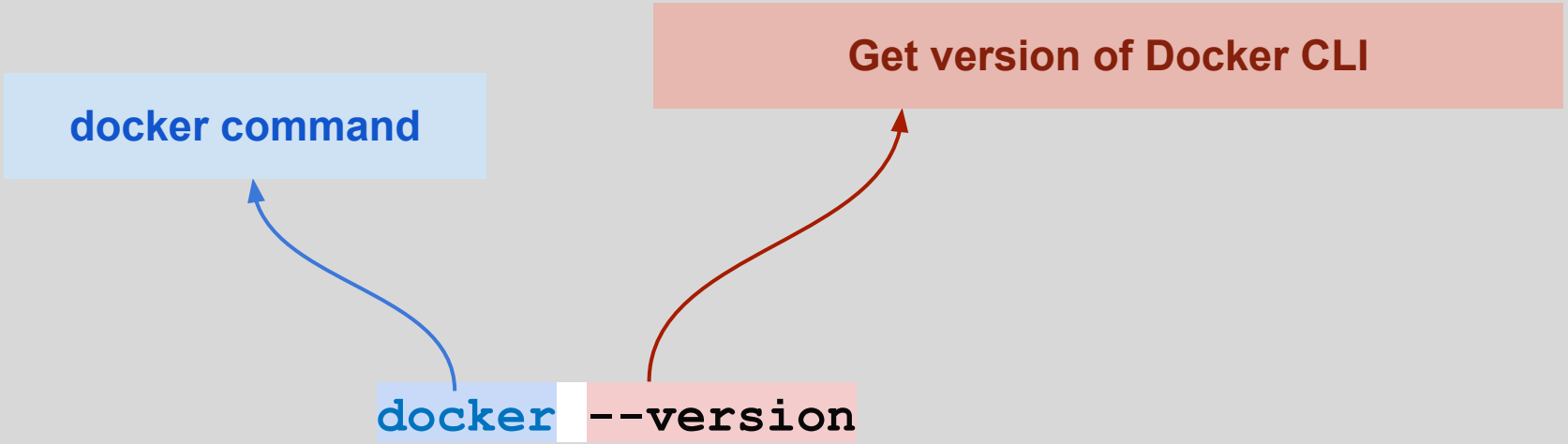
Tutorial: Installing Docker Desktop

- Install Docker Desktop. Use one of the links below to download the proper Docker application depending on your operating system.
  - For Mac users, follow this link- https://docs.docker.com/docker-for-mac/install/.
  - For Windows users, follow this link- https://docs.docker.com/docker-for-windows/install/ Note: You will need to install Hyper-V to get Docker to work.
  - For Linux users, follow this link- https://docs.docker.com/install/linux/docker-ce/ubuntu/
- Once installed run the docker desktop.
- Open a Terminal window and type `docker run hello-world` to make sure Docker is installed properly.



Tutorial: Docker commands

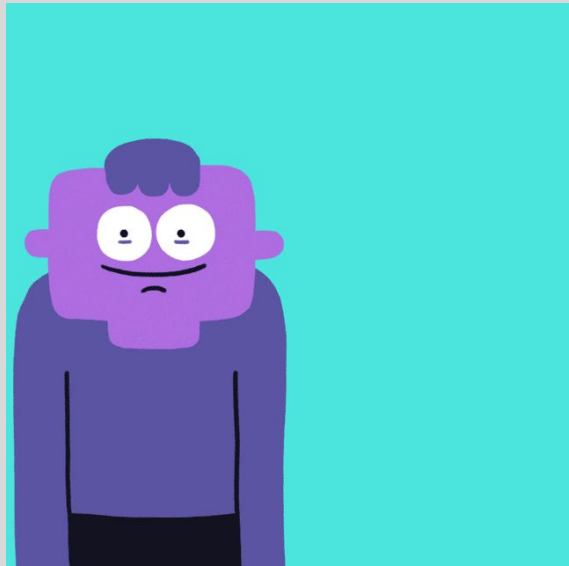
Check what version of Docker



Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)

git clone https://github.com/dlops-io/simple-translate



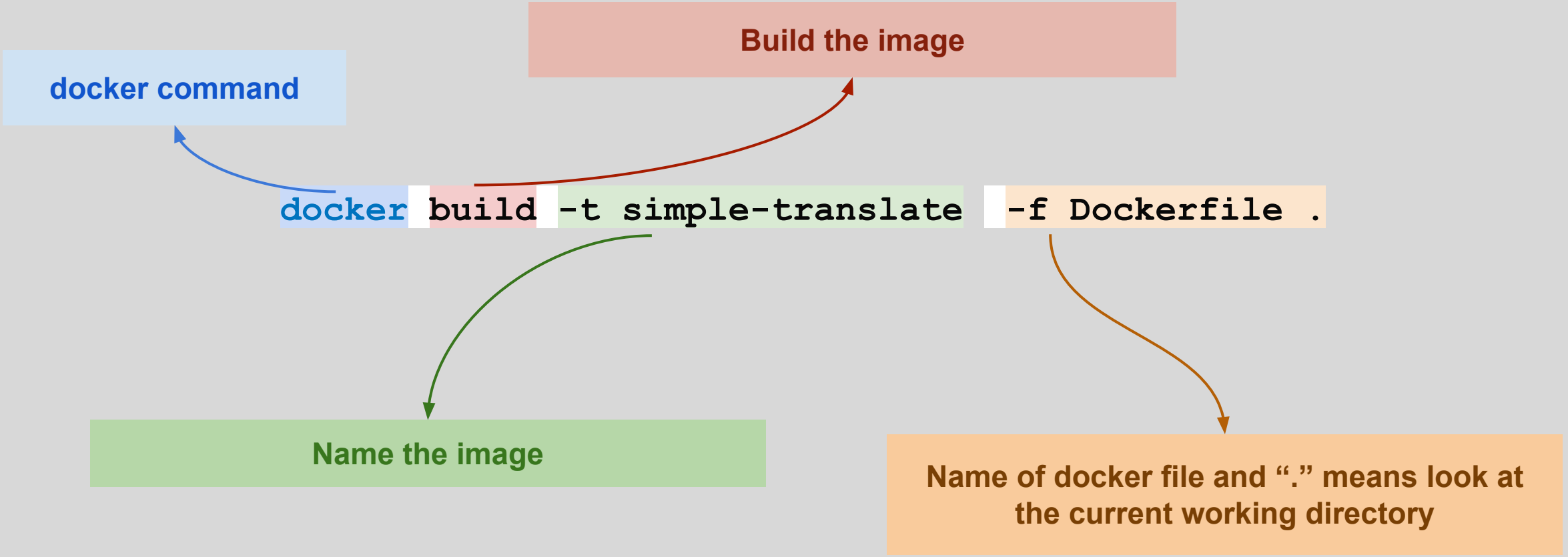
Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)
  - Build a container



Tutorial: Docker commands

Build an image based on a Dockerfile



Dockerfile

```
# Use the official Debian-hosted Python image
FROM python 3.12-slim-bookworm

# Tell uv to copy packages from the wheel into the site-packages
# ENV UV_LINK_MODE=copy
# ENV UV_PROJECT_ENVIRONMENT=/home/app/.venv

# Ensure we have an up to date baseline, install dependencies
# apt-get is a command-line tool used to manage packages
RUN set -ex; \    # -e build process will stop if any command following set -ex fails. -x prints the output
   apt-get update && \   # updates the local package index
   apt-get upgrade -y && \  # upgrade all the installed packages
   apt-get install -y --no-install-recommends build-essential git && \
   pip install --no-cache-dir --upgrade pip && \
   pip install uv

# Add Source code
ADD . /  # adds the content of the current directory "." into the root directory of the container
RUN uv sync

# Entry point
ENTRYPOINT ["/bin/bash"]

# Get into the shell
CMD ["-c", "source /home/app/.venv/bin/activate && exec bash"]
```

Docker Image as Layers

```
>docker build -t simple-translate  -f Dockerfile .

 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 756B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim-buster                                          1.0s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim-buster@sha256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935   2.8s
 => => resolve docker.io/library/python:3.11-slim-buster@sha256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935   0.0s
 => => sha256:81b2c804d9ba5014835bedffff61fb42e5d78be661b781654cda06b3d95237f0 1.37kB / 1.37kB                      0.0s
 => => sha256:12cacc23b6dec78ca7b056d56e3de48252669ed49fffd95ed36adbf9dfe3cec0 6.85kB / 6.85kB                      0.0s
 => => sha256:d191be7a3c9fa95847a482db8211b6f85b45096c7817fdad4d7661ee7ff1a421 25.92MB / 25.92MB                    1.4s
 => => sha256:14aea17807c4c653827ca820a0526d96552bda685bf29293e8be90d1b05662f6 2.65MB / 2.65MB                      0.6s
 => => sha256:67cefd826e1d4a3bce3c47a040ab445ba7ba6586dea8b8380bdad6ee3462f9c1 12.10MB / 12.10MB                    1.3s
 => => sha256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935 988B / 988B                          0.0s
 => => sha256:195c388ea91b233c774667795edf5a47d3b02b3db8da49447d964dbafee7a786 244B / 244B                          0.7s
 => => sha256:db8899040fb5395274edb3f6930ed67e7c7a4cd70adc8f6f21cfa2ab92dce912 3.38MB / 3.38MB                      1.2s
 => => extracting sha256:d191be7a3c9fa95847a482db8211b6f85b45096c7817fdad4d7661ee7ff1a421                           0.8s
 => => extracting sha256:14aea17807c4c653827ca820a0526d96552bda685bf29293e8be90d1b05662f6                           0.1s
 => => extracting sha256:67cefd826e1d4a3bce3c47a040ab445ba7ba6586dea8b8380bdad6ee3462f9c1                           0.3s
 => => extracting sha256:195c388ea91b233c774667795edf5a47d3b02b3db8da49447d964dbafee7a786                           0.0s
 => => extracting sha256:db8899040fb5395274edb3f6930ed67e7c7a4cd70adc8f6f21cfa2ab92dce912                           0.2s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 161.76kB                                                                               0.0s
 => [2/4] RUN set -ex;        apt-get update &&    apt-get upgrade -y &&        apt-get install -y --no-install-recommends build-essential git &&  p   24.0s
 => [3/4] ADD . /                                                                                                   0.0s
 => [4/4] RUN uv sync                                                                                               7.8s
 => exporting to image                                                                                              0.7s
 => => exporting layers                                                                                             0.7s
 => => writing image sha256:e473d8916478a1f09ecfeba01dde5113133490541018fb34dba99947ff140ba0                        0.0s
 => => naming to docker.io/library/simple-translate                                                                 0.0s
```

Step1: Instruction 1

Step2: Instruction 2

Step3: Instruction 3

Step4: Instruction 4

Docker Image as Layers

```
> docker images
REPOSITORY        TAG       IMAGE ID       CREATED          SIZE
hello_world_cmd   latest    e473d8916478   47 hours ago     8.83MB
simple-translate  latest    e473d8916478   22 minutes ago   483MB
```

```
> docker image history simple-translate
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
484239a9b64d   28 hours ago   CMD ["/bin/bash"]                               0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ENTRYPOINT ["uv" "run"]                         0B        buildkit.dockerfile.v0
<missing>      28 hours ago   RUN |1 DEBIAN_PACKAGES=build-essential curl …   26.9MB    buildkit.dockerfile.v0
<missing>      28 hours ago   COPY --chown=app:app . ./ # buildkit            1.25MB    buildkit.dockerfile.v0
<missing>      28 hours ago   WORKDIR /app                                    4.1kB     buildkit.dockerfile.v0
<missing>      28 hours ago   USER app                                        0B        buildkit.dockerfile.v0
<missing>      28 hours ago   RUN |1 DEBIAN_PACKAGES=build-essential curl …   396MB     buildkit.dockerfile.v0
<missing>      28 hours ago   ENV UV_PROJECT_ENVIRONMENT=/home/app/.venv      0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ENV UV_LINK_MODE=copy                           0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ENV PYTHONUNBUFFERED=1                          0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ENV DEBIAN_FRONTEND=noninteractive              0B        buildkit.dockerfile.v0
<missing>      28 hours ago   ARG DEBIAN_PACKAGES=build-essential curl lib…   0B        buildkit.dockerfile.v0
<missing>      3 weeks ago    CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      3 weeks ago    RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      3 weeks ago    RUN /bin/sh -c set -eux;  savedAptMark="$(a…    44.9MB    buildkit.dockerfile.v0
<missing>      3 weeks ago    ENV PYTHON_SHA256=c30bb24b7f1e9a19b11b55a546…   0B        buildkit.dockerfile.v0
<missing>      3 weeks ago    ENV PYTHON_VERSION=3.12.11                      0B        buildkit.dockerfile.v0
```

Why Layers

Why build an image with multiple layers when we can just build it in a single layer?
Let's take an example to explain this concept better, let us try to change the Dockerfile_cmd we created and rebuild a new Docker image.

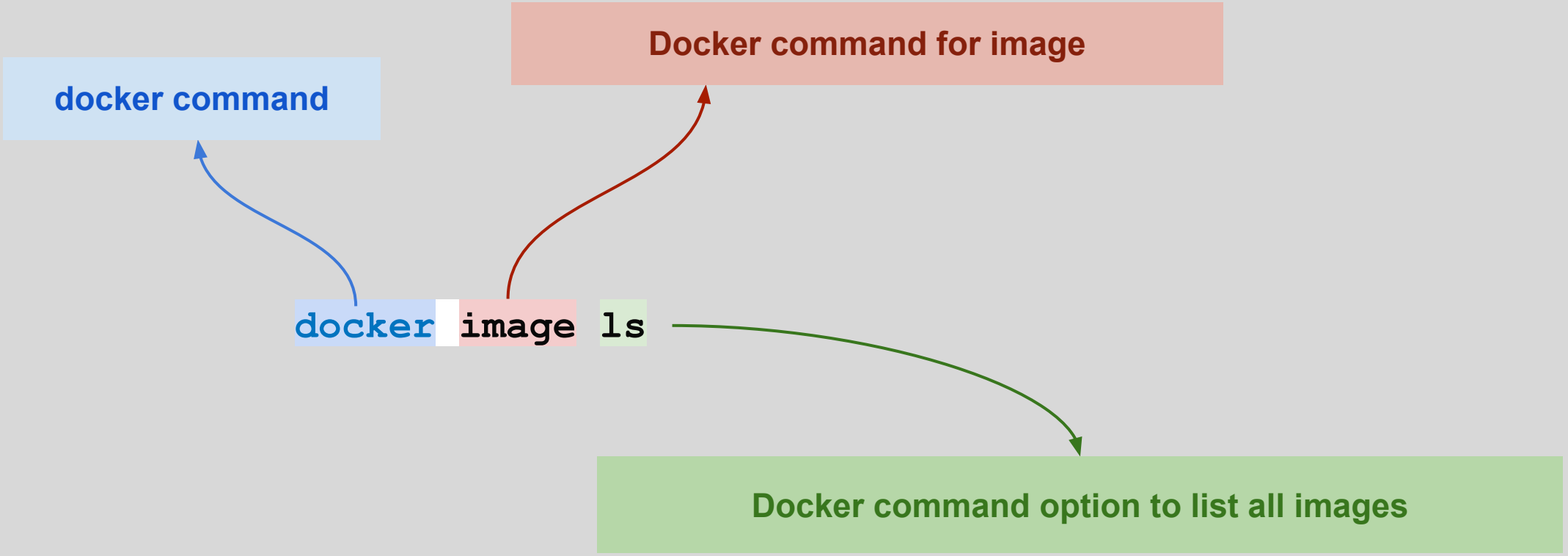
```
> docker build -t hello_world_cmd -f Dockerfile_cmd .
[+] Building 0.6s (9/9) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                 0.0s
 => => transferring dockerfile: 756B                                                                                                 0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim-buster                                                           0.5s
 => [internal] load .dockerignore                                                                                                    0.0s
 => => transferring context: 2B                                                                                                      0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim-bust                 a256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935  0.0s
 => [internal] load build context                                                                                                    0.0s
 => => transferring context: 5.91kB                                                                                                  0.0s
 => CACHED [2/4] RUN set -ex;   apt-get update &&   apt-get upgrade -y &&       apt-get install -y --no-install-recommends build-essential git &&  0.0s
 => CACHED [3/4] ADD . /                                                                                                             0.0s
 => CACHED [4/4] RUN uv sync                                                                                                         0.0s
 => exporting to image                                                                                                               0.0s
 => => exporting layers                                                                                                              0.0s
 => => writing image sha256:e473d8916478a1f09ecfeba01dde5113133490541018fb34dba99947ff140ba0                                         0.0s
 => => naming to docker.io/library/simple-translate                                                                                  0.0s
```

Have seen this before. Use cache

As you can see that the image was built using the existing layers from our previous docker image builds. If some of these layers are being used in other containers, they can just use the existing layer instead of recreating it from scratch.

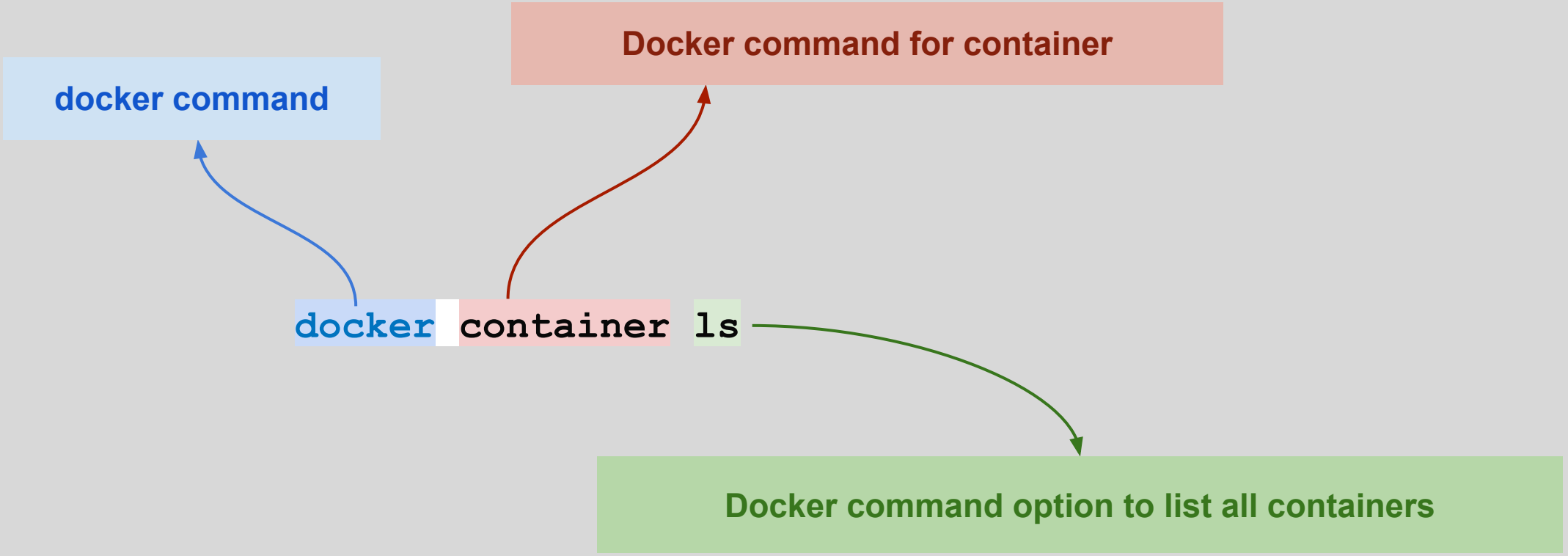
Tutorial: Docker commands

List all docker images



Tutorial: Docker commands

List all running docker containers



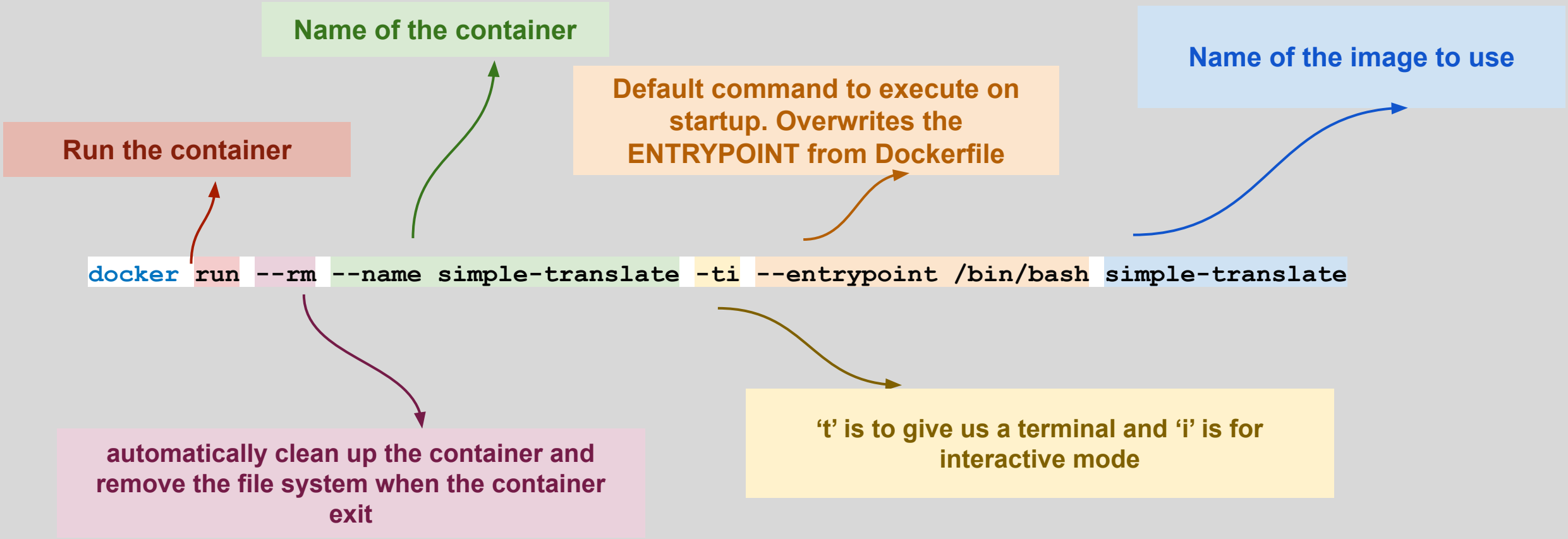
Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)
  - Build a container
  - Run a container



Tutorial: Docker commands

Run a docker container using an image



Tutorial: Docker commands

Open another command prompt and check how many container and images we have

```
docker image ls
```

```
docker container ls
```

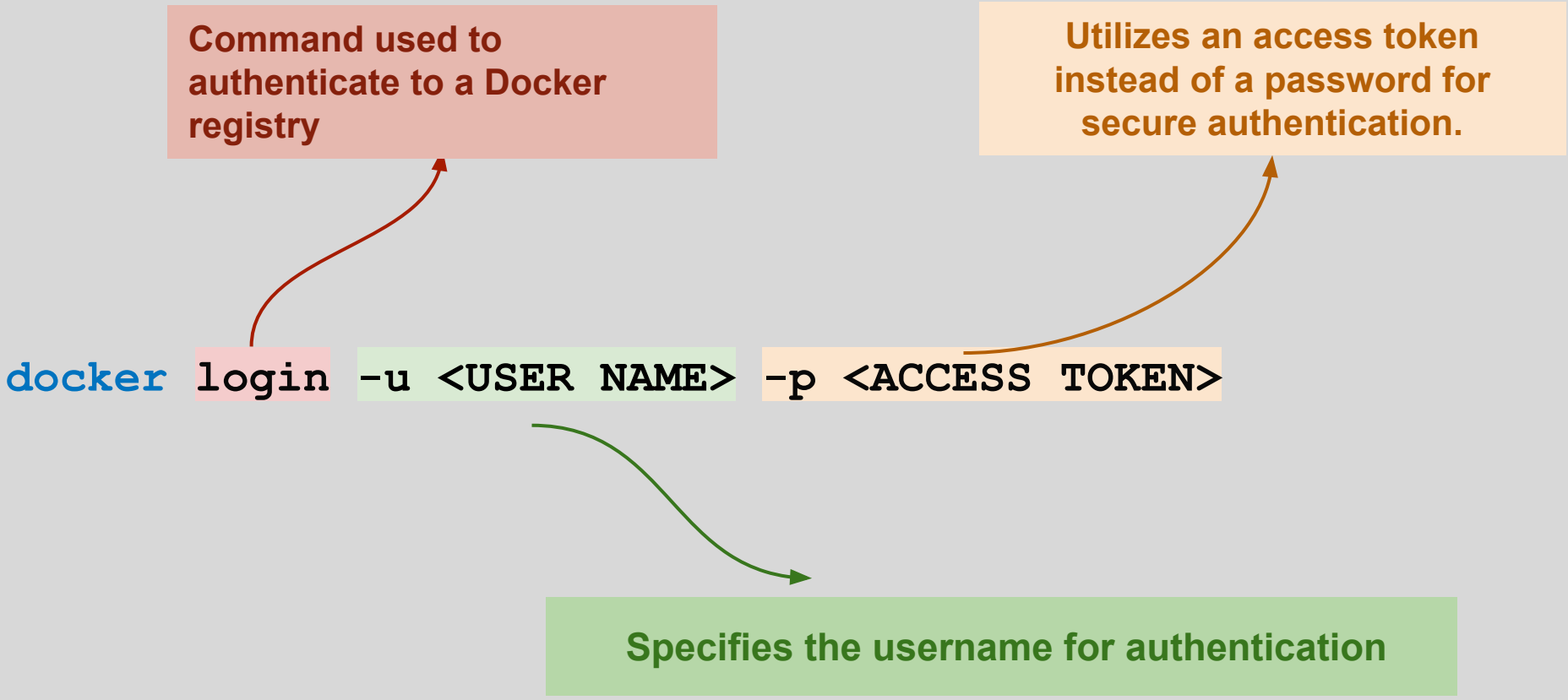
Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)
  - Build a container
  - Run a container
  - Push container on Docker Hub



Tutorial: Docker commands

Sign up in Docker Hub and create an Access Token. Use that token to authenticate with the command below



Tutorial: Docker commands

Tag the Docker Image

Assigns a new name or tag to an image

Docker Hub username or repository namespace.
The new name for the tagged image.

```
docker tag <SOURCE IMAGE NAME>[:TAG] <USER NAME>/<TARGET_IMAGE[:TAG]>
```

The existing image name to be tagged.
Optional tag for the source image (defaults to latest if omitted).

Tutorial: Docker commands

- Push to Docker Hub

Command used to upload a Docker image from your local machine to a remote registry like Docker Hub

The name of the image you want to push to the registry. User name can be included as part of the name

```
docker push <USER NAME>/<TARGET_IMAGE[:TAG]>
```

Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)
  - Build a container
  - Run a container
  - Push container on Docker Hub
  - Pull the new container and run it



Tutorial: Docker commands

- Pull from Docker Hub

Command used to download a Docker image from a registry to your local machine

The name of the image you want to pull and TAG

```
docker pull [OPTIONS] <USER NAME>/<TARGET_IMAGE[:TAG]>
```

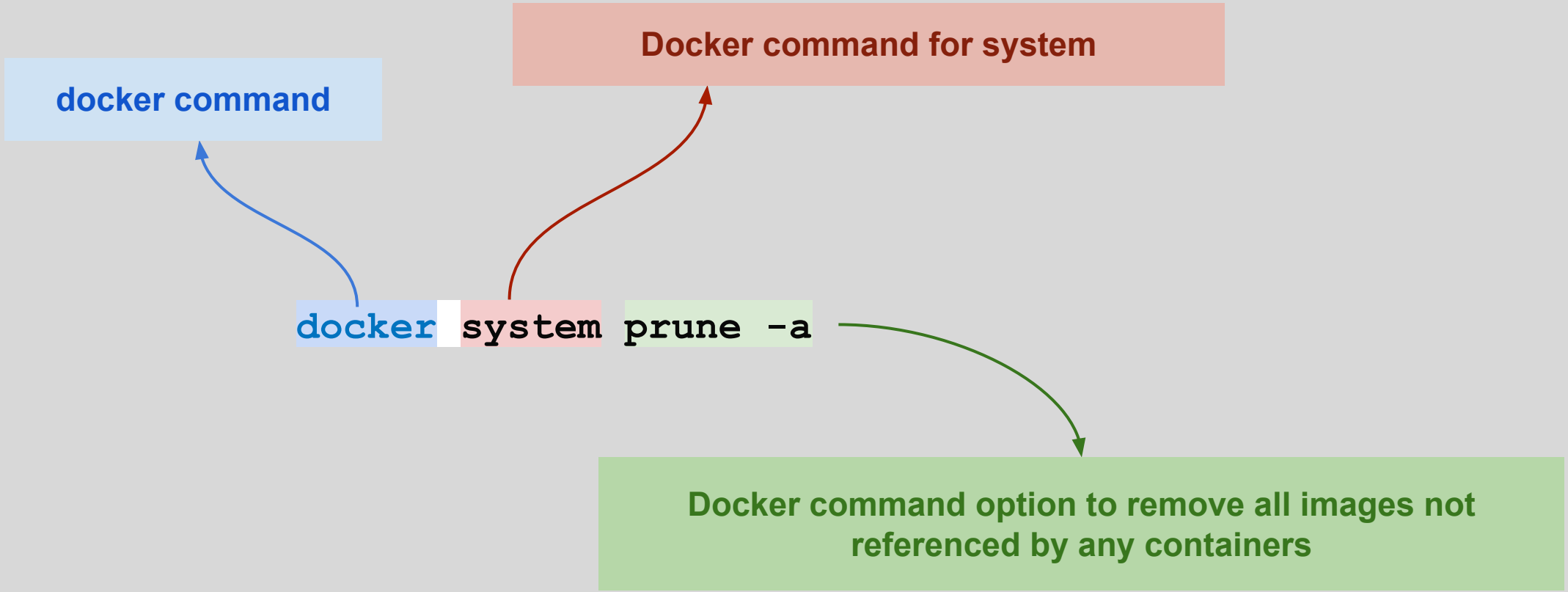
Tutorial: Developing App using Containers

- Let us build the simple-translate app using Docker
- For this we will do the following:
  - Clone or download code (https://github.com/dlops-io/simple-translate)
  - Build a container
  - Run a container
  - Push container on Docker Hub
  - Pull the new container and run it
- For detail instruction go here (https://github.com/dlops-io/simple-translate#developing-app-using-containers-t3)



Tutorial: Docker commands

Exit from all containers and let us clear of all images



Debugging: Troubleshooting Misbehaving Containers

To investigate container issues or crashes, use these four tools: ps, logs, exec, and inspect.

Monitoring Status and Output

- **`docker ps / docker ps -a`:** List running or all containers (including those that have exited with error codes).
- **`docker logs <name>`:** Access stdout/stderr streams. Use the -f flag to follow output in real-time for crash diagnosis.

Debugging: Accessing and Inspecting Containers

Getting a Shell

`docker exec -it <name> /bin/bash`:

Use this command to open a shell within a running container, allowing you to inspect files, check environment variables, and execute commands.

- **Key Distinction:** `docker exec` attaches to an existing, running container, whereas `docker run` initiates a new one.
- **Troubleshooting:** If the container exits immediately, you can debug by overriding the entrypoint: `docker run -it --entrypoint /bin/bash <image>`.

Debugging: Accessing and Inspecting Containers (cont)

Inspecting Container Details

- `docker inspect <name>`: Retrieves the full JSON configuration for the container, including mounts, environment variables, network settings, ports, and the entrypoint.
- **Key Use Cases:**
  - **Mounts:** Verify that volumes are correctly mounted.
  - **Config.Env:** Confirm that specific environment variables are set.
  - **NetworkSettings.Ports:** Check which ports are actively published.

Tutorial: Running App on VM using Docker

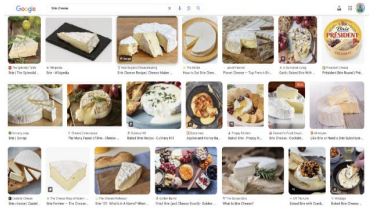
- Let us run the simple-translate app using Docker
- For this we will do the following:
  - Create a VM Instance
  - SSH into the VM
  - Install Docker inside the VM
  - Run the containerized simple-translate app
- Full instructions can be found here (https://github.com/dlops-io/simple-translate#running-app-on-vm-using-docker-t4)



Recap: How do we build an App?

Development

Deployment



Python Script

Python: uv
Chromium: Mac install, Windows install
OS: Mac, Windows



Colab

Python: Colab provided env
OS: Linux

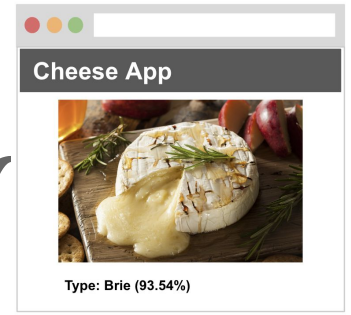
Rest API

Best Model

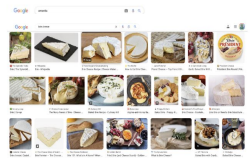


IDE / Code Editor

Python: uv
OS: Mac, Windows



Model Training



Python Script

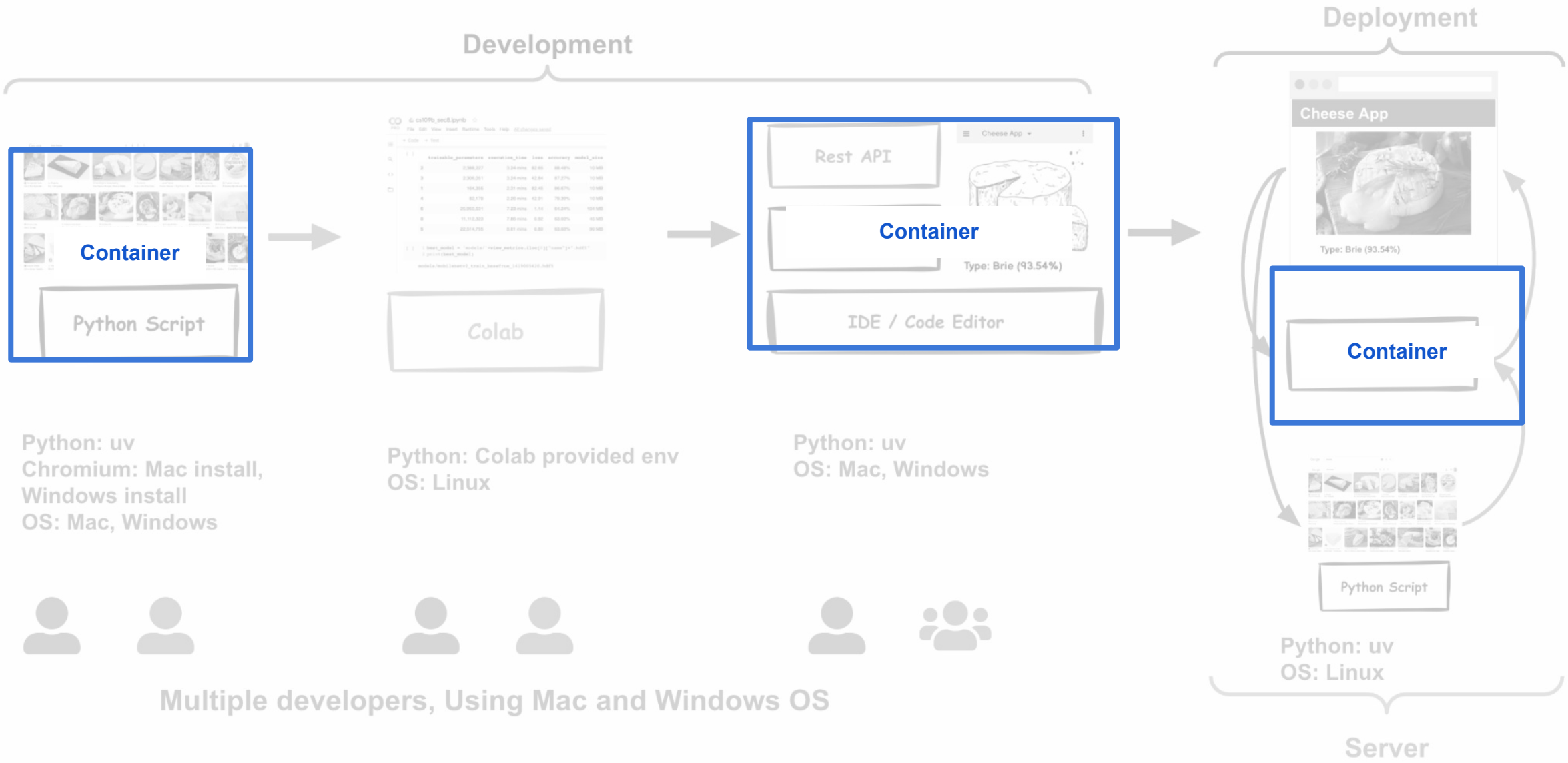
Python: uv
OS: Linux

Server



Multiple developers, Using Mac and Windows OS

Isolate work into containers







Thank you